

API + JSON + FLASK + POSTMAN

Duración: 50 minutos | Trabajo en equipos de 3 como máximo estudiantes

Propósito:

Reconocer las características de las APIs y del formato JSON, comprender la función de Postman y relacionar estas herramientas mediante la creación y prueba de una API sencilla con Flask.

Al finalizar la actividad podrás:

- Distinguir las características básicas de una API.
- Explicar las características del formato JSON y su relación con servicios Web y APIs.
- Identificar las características y funciones principales de Postman.
- Relacionar Postman con el desarrollo y prueba de APIs.
- Levantar y probar una API sencilla desarrollada con Flask.

Organización de la sesión

Actividad	Tema	Tiempo	Producto
1	API y JSON	15 min	Clasificación y explicación
2	Flask: levantar una API	20 min	API funcionando + evidencia
3	Postman: probar la API	15 min	Pruebas + evidencia

ACTIVIDAD 1. ¿API, JSON o comunicación cliente-servidor?

Tiempo: 15 minutos

En equipo, analicen y clasifiquen los siguientes conceptos en tres categorías: A) API, B) JSON y C) comunicación cliente-servidor.

API · Endpoint · Cliente · Servidor · HTTP · GET · POST · JSON · Servicio Web · Recurso · Request · Response · Postman – Recurso – REST- PUT- DELETE- Cliente-Servidor-URL-HTTPS status code-Header-Body-Objeto-Propiedad- Valor-Array-Number-String-Boolean-Null-Servicio Web- Flask-HTML-CSS-PYTHON

Después, respondan en su hoja de trabajo:

1. ¿Qué relación existe entre una API, HTTP y JSON?
2. ¿Cuál es una propiedad del siguiente objeto JSON?
3. ¿Cuál es un valor de texto y cuál es un valor numérico?
4. ¿Qué representan las llaves { } en JSON?
5. ¿Por qué JSON puede utilizarse para intercambiar información entre cliente y servidor?

Reto JSON:

```
{  
  "nombre": "Juan",  
  "edad": 18,
```

}

[illegible]

ACTIVIDAD 2. Levanta tu primera API con Flask- Todos los integrantes de manera colaborativa Tiempo: 20 minutos

Objetivo: levantar un servidor Flask y obtener respuestas en formato JSON.

1. Crea una carpeta

llamada: api_flask/

2. Dentro de la carpeta crea un archivo

llamado: app.py

3. Instala Flask desde la

terminal: `pip install flask`

4. Escribe el siguiente código en

app.py: `from flask import Flask,`

`jsonify`

`app = Flask(__name__)`

`@app.route("/")`

`) def inicio():`

`return jsonify({`

`"mensaje": "API`

`funcionando", "curso":`

`"Desarrollo de APIs"`

`})`

`@app.route("/`

`alumnos") def`

`alumnos():`

`return jsonify([`

`{`

`"id": 1,`

`"nombre": "Ana"`

`},`

`{`

`"id": 2,`

`"nombre": "Carlos"`

`}`

`])`

`if __name__ == "__main__":`

`app.run(debug=True)`

5. Ejecuta el servidor:

`python app.py`

6. Verifica en la terminal que Flask indique que el servidor está disponible, por ejemplo: `http://127.0.0.1:5000`

```
Desarrollo/apls postman/api_Flash via v2.14.7
> ./venv/bin/python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
PRESS CTRL+C TO QUIT
* Restarting with stat
* Debugger is active!
* Debugger PIN: 111-111-111
127.0.0.1 - - [02/Oct/2020 12:47:13] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [02/Oct/2020 12:47:13] "GET /alumnos HTTP/1.1" 200 -
127.0.0.1 - - [02/Oct/2020 12:47:13] "GET /materias HTTP/1.1" 200 -
127.0.0.1 - - [02/Oct/2020 12:47:20] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [02/Oct/2020 12:51:04] "GET /alumnos HTTP/1.1" 200 -
127.0.0.1 - - [02/Oct/2020 12:55:28] "GET /materias HTTP/1.1" 200 -
```

Programación Rodas > Primer petición

GET Send

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Cookies Headers 5 Test Results 200 OK - 4 ms - 235 B Save Response

{ JSON Preview Visualize

```
1 {
2   "curso": "Desarrollo de APIs",
3   "mensaje": "API funcionando"
4 }
```

GET

https://127.0.0.1:5000/usuarios

Send

Docs

Params

Authorization

Headers

Body

Scripts

Settings

Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body

Cookies

Headers

Test Results

200 OK

4 ms

257 B

Save Response

JSON

Preview

Visualize

```
1  [
2    {
3      "id": 1,
4      "nombre": "Ana"
5    },
6    {
7      "id": 2,
8      "nombre": "Carlos"
9    }
10 ]
```

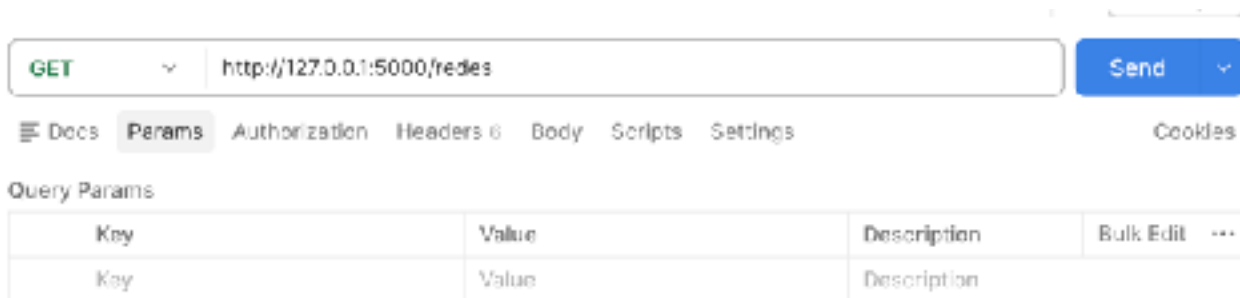
- Prueba en el navegador:

http://127.0.0.1:5000/ http://

127.0.0.1:5000/alumnos

Responde:

6. Agrega una nueva ruta /redes donde salude



- ¿Cuál es el endpoint /alumnos? http://127.0.0.1:5000/alumnos
-
- ¿Qué método HTTP estamos utilizando? GET, porque solo estamos consultando información (es el método que usa el navegador al abrir una URL y el que Flask acepta por defecto en una ruta).
- ¿Qué información devuelve el endpoint /alumnos y el endpoint de redes? /alumnos` devuelve un arreglo JSON con dos objetos, cada uno con `id` y `nombre`

Producto: Captura de pantalla donde se observe el servidor Flask funcionando y una respuesta JSON.

```

WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 111-314-688
127.0.0.1 - - [02/Oct/2026 12:47:13] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [02/Oct/2026 12:47:13] "GET /alumnos HTTP/1.1" 200 -
127.0.0.1 - - [02/Oct/2026 12:47:15] "GET /redes HTTP/1.1" 200 -
127.0.0.1 - - [02/Oct/2026 12:47:29] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [02/Oct/2026 12:51:54] "GET /alumnos HTTP/1.1" 200 -
127.0.0.1 - - [02/Oct/2026 12:55:28] "GET /redes HTTP/1.1" 200 -
127.0.0.1 - - [02/Oct/2026 13:03:51] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [02/Oct/2026 13:07:20] "GET /alumnos HTTP/1.1" 200 -

```

ACTIVIDAD 3. Prueba tu API con Postman

Tiempo: 15 minutos

Objetivo: utilizar Postman para enviar solicitudes a la API creada con Flask y analizar sus respuestas.

11. Abre Postman.
12. Crea una nueva petición utilizando el método GET.
13. Escribe la URL: `http://127.0.0.1:5000/`
14. Presiona Send y observa la respuesta.
15. Crea una segunda petición GET con la URL: `http://127.0.0.1:5000/alumnos`
16. Presiona Send y analiza la

respuesta JSON. Identifica en Postman:

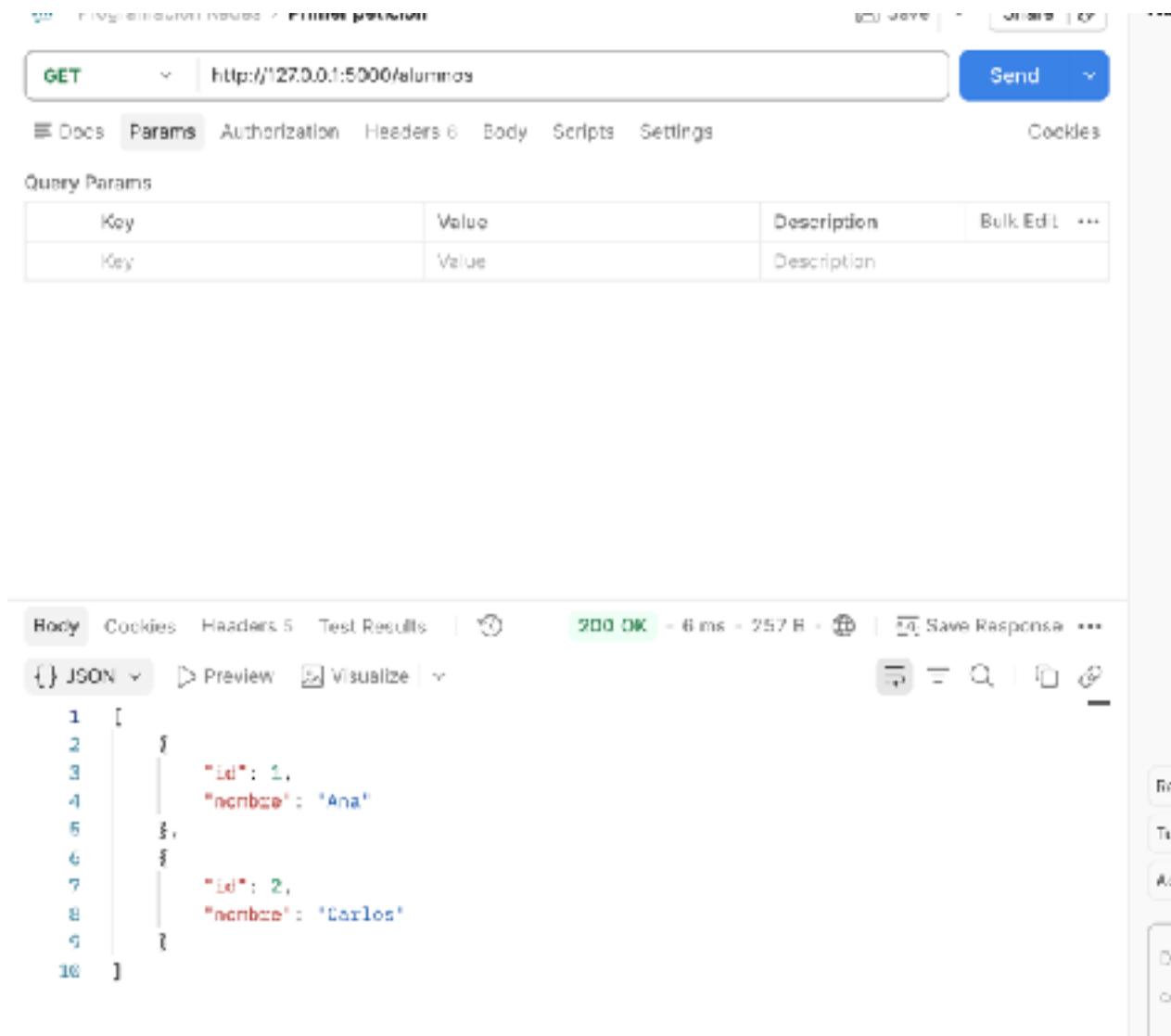
Elemento	¿Qué función cumple?
GET	Solicitar o recuperar datos
URL	Direction unica end point
Send	Ejecuta acción
Response	Respuesta del servidor
Status Code	Coding numerico
Body	Cuerpo de la solicitud
Headers	Cabecera que contiene metadatos

Reto final:

Explica con tus propias palabras: ¿qué ventaja tiene Postman respecto a probar una API solamente desde el navegador?

Desde la barra del navegador solo se pueden hacer peticiones GET. Con Postman se puede usar cualquier método (POST, PUT, DELETE), mandar datos en el body y headers propios, y ver con claridad el status code, los headers y el tiempo de respuesta. Además, las peticiones se pueden guardar en colecciones para repetir las pruebas.

Producto: Captura de pantalla de Postman donde aparezca GET /alumnos, el código 200 OK y la respuesta JSON. **CIERRE. El API en 60 segundos**



Tiempo: 5 minutos

Participa en la puesta en común. El docente podrá seleccionar estudiantes o equipos para responder:

- ¿Qué es una API y para qué sirve? Es un conjunto de reglas y endpoints que permite que dos programas se comuniquen; sirve para que un cliente pida datos o servicios a un servidor sin conocer cómo está hecho por dentro.
- ¿Qué es JSON y por qué se utiliza en las APIs? Es un formato de texto para representar datos con pares propiedad-valor; se usa porque es ligero, legible y lo entiende cualquier lenguaje
-
- ¿Para qué sirve Postman en el desarrollo de una API? Para enviar peticiones a la API y revisar sus respuestas, es decir, para probarla sin tener que programar un cliente.
-
- ¿Qué relación existe entre Flask y Postman? Postman actúa como cliente: envía las peticiones y permite inspeccionar lo que Flask responde.
-

Idea clave: Flask crea y ejecuta la API → Postman envía peticiones → Flask responde → Postman permite inspeccionar la respuesta.

Evaluación de la sesión

Actividad	Evidencia	Puntos
1. API + JSON	Clasificación + explicación	2
2. Flask	API funcionando	5
3. Postman	Peticiones y respuestas	3
Total		10

Roles sugeridos dentro del equipo

Rol	Responsabilidad
Analista	Resuelve y explica los conceptos de API y JSON.
Desarrollador	Levanta y verifica la API con Flask.
Tester	Realiza las pruebas con Postman.
Documentador	Registra respuestas y evidencias.

Importante: cambien los roles durante la sesión para que todos los integrantes participen.